

llm-train CC12 Grouped-GEMM Selector 兼容性事件报告提纲

从 production authority 失败到可复核修复

事件研究提纲，拟作为独立技术报告或系统论文案例研究

2026 年 8 月 11 日

摘要

这份报告聚焦 TrainVerify 建立真实双 GPU authority 时确认的一项 llm-train runtime/hardware selector 兼容性问题。固定版本的 llm-train 在 compute capability 12.0 的 RTX PRO 6000 Blackwell 上，把 grouped-GEMM/Triton 选择送入不适用的 B200-only candidate set，导致 production 双 rank compile 路径无法完成。调查将失败收窄到硬件分派条件，并使用一个字节与来源都绑定的最小补丁，让 CC12 回退到 generic A100/H100 Triton autotune candidates。报告同时公开两条撤回记录：cache K/V ownership 的错误归因，以及 balance-loss 语义 bug 的越界推断。它们不是已确认的 llm-train bug。本提纲规定事件时间线、复现、根因、修复、验证和证据边界，避免把兼容性故障扩大成训练语义错误。

报告定位

- **确认的事件：**llm-train grouped-GEMM/Triton selector 在 CC12 production hardware 上选择了不兼容的候选集合，阻断真实双 rank compile。
- **没有确认的结论：**没有证据表明 grouped-GEMM 的数学算子值错误，也没有证据据此推断模型训练语义错误。
- **分开的另一项 finding：**nnScaler RVD/annotation/adapter 无法表达 post-shuffle zigzag ownership，只完成 CPU/Gloo 双进程源码复现，不能写成 CUDA/NCCL 复现。
- **已撤回：**“cache K/V 是 zigzag-owned”和“balance-loss mask 与 routing map 错位”都不得作为 llm-train bug 结论。

1 执行摘要

本节任务：让维护者不用读调查细节也能知道发生了什么、受什么影响、如何修、验证到了哪一层。

建议用一页事件卡片：

字段	内容
受影响路径	llm-train production compile 中的 grouped-GEMM/Triton launch configuration selection

触发条件	compute capability 12.x; 真实双 GPU、双 rank authority path
用户可见症状	production authority compile 失败, 无法生成可验证 SM/PM 工件
根因位置	固定 llm-train revision 的 llm/kernel/gemm.py candidate selector
最小修复	将两处 CC >= 10 专用分支条件收窄为 CC == 10
修复效果	CC12 使用 generic A100/H100 Triton autotune candidate set
不变项	operator 数学定义、model code、graph annotation、reviewed base revision
验证	双 rank authority 成功; 补丁身份和 patched source hash 进入 receipts/provenance
残余风险	尚需独立性能回归、更多 CC12 设备和上游维护者确认

表述边界: 标题、摘要和结论统一使用“runtime/hardware selector 兼容性问题”。不得简写成“Train-Verify 证明 llm-train 训练结果错误”。

2 系统背景

本节任务: 解释为什么一个 kernel selector 会出现在形式化验证 authority 链里, 以及普通 smoke test 为什么看不到它。

2.1 llm-train、nnScaler 与 TrainVerify 的边界

llm-train 提供 YOCO 模型和 grouped-GEMM kernel 路径; nnScaler trace、规划并生成 SM/PM; TrainVerify 要求从真实 production path 固化 authority 后再翻译为 Lean。用一张三层图说明职责。

2.2 Grouped GEMM 在 YOCO MoE 中的位置

用一个 MoE layer 的数据流图解释 router、token dispatch、expert-local up/gate/down projection、grouped GEMM、activation 和 AllToAll。只介绍理解 selector 所需部分, 不在事件报告中重讲整个模型。

2.3 Kernel selector 与数学算子的区别

说明 operator 定义回答“计算什么”, selector 回答“在当前硬件上使用哪组 launch/autotune candidates”。selector 失败可以阻断编译或运行, 但本事件没有显示数学公式发生变化。

2.4 为什么 CPU smoke 和 planner-only test 不够

CPU smoke 能检查 import、trace、code generation、pickle extraction 和 shuffle wiring; 它不进入真实 CC12 Triton selector、GPU profile 和双 rank NCCL 路径。仅设置 plan_ngpus=2 也不等于真实启动两张 GPU。

图表计划：图 1：训练源码、nnScaler compile、grouped-GEMM selector、GPU kernel 和 TrainVerify authority 的调用链；用红色标出失败位置。

3 环境与 revision 范围

本节任务：把每项结论绑定到实际检查过的源码、硬件和运行环境，防止跨 revision 外推。

CC12 事件固定在 llm-train 9a1be1d5fd1c063d80be82797692cdc7d23cfbef、nnScaler d3d468ed23edb2f28aa8566b2dfb6ed49c5955cf，硬件为两张 NVIDIA RTX PRO 6000 Blackwell Workstation Edition, compute capability 12.0, CUDA runtime 12.8, NCCL 2.27.3, driver 595.71.05。模型配置为 YOCO-MoE-A0.4B、24 layers、sequence length 4096。

历史 nnScaler zigzag/RVD finding 使用另一组 revision：nnScaler 1102e629ee68ab6f8f4a7c2e721ea894e5962131 与 llm-train 30b80f546d46aacbf8316c983550c50a56bcd1ac。该 finding 不能自动外推到 CC12 authority 所用版本，也不能与 CC12 root cause 合并。

图表计划：表 1：每条 finding 的 llm-train、nnScaler、TrainVerify revision，硬件路径，复现层级和当前证据状态。

4 事件检测

本节任务：按证据发生顺序记录现象，不用事后知识重写成“一眼看出根因”。

4.1 预期行为

固定 llm-train、nnScaler、TrainVerify revision，真实启动两个 rank，在两张 RTX PRO 6000 Blackwell 上完成 production SM/PM compile，并生成带 hardware、profile、rank receipts 和 pickle hashes 的 authority。

4.2 实际失败

记录第一条 production torchrun 失败的命令、时间、机器、GPU inventory、软件版本、rank、exit code、完整 stderr 和第一个稳定 stack frame。这里必须引用原始日志；若日志未保存，则明确标记为不可恢复证据，不能补写一个“合理的”报错。

4.3 为何把它当作 authority BLOCK

CPU smoke 通过不允许降级替代 production authority。失败发生在真实双 rank compile 路径，因此正确动作是停止 authority 发布并定位硬件 dispatch，而不是手写 PM graph、跳过 profile 或修改 Lean statement。

所需证据：需要找到或明确缺失：首次失败日志、GPU inventory、CUDA、Triton、PyTorch、driver、NCCL 版本、rank receipts，以及失败前后的环境清单。

5 诊断时间线

本节任务：把每一步假设、实验、结果和下一步决策连起来。

版本库能够确认的里程碑为：2026-07-29 修正 nnScaler RVD 归因；2026-07-30 完成 CPU/Gloo 双进程真实源码复现；2026-08-06 落地 CC12 selector 修复与 provenance attestation；2026-08-08 撤回 cache K/V zigzag 归因；2026-08-11 重写公开审计说明。2026-08-06 是修复落地日期，不得冒充首次故障发生日期；首次失败的原始时间仍需从运行日志恢复。

建议同时采用下表记录 CC12 诊断内部顺序，成稿时为 T0 至 T4 补真实时间戳：

时间	观察	假设	实验/证据	结论
T0	双 rank production compile 失败	图或模型逻辑错误	与 CPU smoke、planner-only 对照	失败只在真实 GPU path 出现
T1	authority metadata 显示 CC12	硬件 dispatch 不支持	固定环境并检查 selector branch	优先调查 CC 分派
T2	失败收敛至 gemm.py	candidate set 与硬件不匹配	列出 CC 条件和候选来源	CC12 进入 B200-only set
T3	最小条件补丁	改 selector 即可恢复	只改两处条件，其他源码冻结	候选集合改变，算子接口不变
T4	双 rank 重跑成功	修复充分	生成 authority 与 receipts	production path 恢复

每一步都要记录被排除的替代解释，例如缺 profile、nnScaler DP solver、CUDA/NCCL 初始化、模型图错误和 TrainVerify emitter 错误。

6 根因分析

本节任务：从源码条件、硬件类别和候选集合三方面说明为什么失败，而不是只展示补丁。

6.1 触发条件

给出固定 llm-train revision、文件和两处 selector 条件。解释原条件如何把所有 CC ≥ 10 设备送入专用分支，而该分支实际对应 B200-only candidate set。

6.2 为何 CC12 不满足该分支假设

需要由实际候选定义、Triton/backend 约束和原始失败日志共同支持。不能只因为补丁后成功，就反推每个候选为何失败。无法从现有证据确定的低层原因明确留空。

6.3 五个为什么

建议按以下链条展开：

1. 为什么 production compile 失败？因为 grouped-GEMM selector 没有找到可用的 launch candidate，或在候选编译阶段失败。
2. 为什么选择了这组 candidates？因为 CC12 满足 $CC \geq 10$ 。
3. 为什么这条条件不正确？因为分支能力边界对应 CC10/B200，而不是所有未来更高 compute capability。
4. 为什么测试没有提前发现？现有测试没有覆盖 CC12 production 双 rank 路径。
5. 为什么形式化工作触发了它？authority policy 禁止用 CPU 或模拟图替代真实编译，迫使系统进入此前未覆盖的硬件路径。

表述边界：若原始日志只能证明“候选编译失败”，正文就写到这一层；不要补成特定 PTX、寄存器或数值故障。

图表计划：图 2：原 selector decision tree 与修复后的 decision tree；表 1：CC、设备类别、候选集合和已测试状态。

7 最小修复

本节任务：说明补丁为什么最小、改变了什么、没有改变什么，以及如何防止补丁漂移。

7.1 补丁内容

scripts/yoco_regen/patch_llm_cc12_gemm.py 将 llm/kernel/gemm.py 中两处 $CC \geq 10$ 收窄为 $CC == 10$ 。补丁标识为：

```
cc12_generic_triton_fallback_v1
```

7.2 为何选择 fallback 而不是改模型或图

补丁只改变 launch configuration selection。它不改变 grouped-GEMM 的 tensor contract、模型调用、nnScaler annotation 或 SM/PM topology，因此不会通过“修改被验证程序的数学含义”来绕开失败。

7.3 补丁完整性

说明精确替换数、原始字节预期、幂等性、半补丁状态拒绝、patched-file SHA-256 和 rank receipt 绑定。正式运行从 fixed revision 的 private materialization 产生 patched source，不修改开发 worktree。

8 验证与回归

本节任务：分别验证补丁脚本、production 执行、图产物和性能，避免“跑通一次”被写成完整修复证明。

8.1 补丁脚本测试

仓库已有 `test_cc12_gemm_patch_is_exact_and_idempotent` 和 `test_cc12_gemm_patch_rejects_partial_source`，验证精确两处替换、幂等性和 `partial source` 拒绝。成稿前还应补充错误 `revision`、零命中、`marker` 被篡改和文件存在额外修改的独立测试。该层只能证明文本补丁机制，不能替代 CC12 GPU runtime 复现。

8.2 真实双 GPU 验证

在两张 RTX PRO 6000 Blackwell、双 rank NCCL 上重跑 `production authority`。报告实际命令、环境、`exit code`、`rank receipts` 和 `authority completion`。

8.3 产物验证

确认 SM/PM graph、节点数、`raw digest`、`hardware profile`、`patched source hash` 和补丁标识进入 `provenance`。若修复前没有可比较的完整 `authority`，不得声称“补丁前后 `graph byte-identical`”；只能说明成功产物与预期 `canonical graph` 一致。

8.4 性能验证

需要补测 `generic fallback` 对编译时间、`autotune` 时间和 `grouped-GEMM runtime` 的影响，并与可用的 CC12 baseline 对照。当前没有数据时写成 `open action`，不得说“无性能损失”。

8.5 跨硬件回归

至少覆盖 CC10 原分支、CC12 fallback 和较低 CC 的既有分支，确认条件收窄没有把 CC10 从专用路径移走。能够访问的硬件和仅做静态检查的范围要分开报告。

图表计划：表 2：修复验证矩阵；表 3：`production authority` 结果；图 3：`compile/runtime` 性能对比。

9 影响评估

本节任务：限定受影响条件、故障类型和用户影响。

- **直接影响：**在受影响硬件与固定源码路径上，`production graph compile/authority generation` 被阻断。
- **间接影响：**无法生成真实 `authority` 时，任何后续 `Lean proof` 都不能合法绑定该硬件路径。
- **未建立的影响：**没有证据显示已完成训练产生错误模型参数或 `grouped-GEMM` 数值悄悄错误。
- **版本范围：**只对检查过的 `llm-train revision` 和实际 `selector source` 下结论；上游其它版本需单独审计。

需要在成稿前补充受影响配置矩阵和上游是否已有等价修复。

10 两个撤回，以及为什么必须写进报告

本节任务：展示调查如何纠正错误假设，防止未来把候选分析当成已确认 bug。

10.1 撤回一：cache K/V ownership

早期看到 decoder stream 在 shuffle 后保持 zigzag，一度猜测 cache K/V 也来自 zigzag-owned hidden state。重新沿 `model.py` producer 顺序审计后发现，K/V projection 在 `wrap_maybe_shuffle(h)` 之前完成；K/V 是普通连续 shard，Q 和 decoder stream 才是 shuffle 后 zigzag。建立在错误假设上的候选 commits 不进入 authority，也不构成 llm-train bug 证据。

10.2 撤回二：balance-loss 语义 bug

早期调查一度认为 routing map 与 position-indexed loss mask 错位，但目标 TID 是 leaf output，balance-loss consumer 位于 trace scope 之外。用 scope 外代码推断 scope 内 goal 为假属于方法错误。除非固定 revision、源码位置和可复现实验重新建立证据，不得恢复这项结论。

10.3 与 nnScaler zigzag/RVD finding 的边界

另一个已确认 finding 是 nnScaler RVD/annotation/adaptor integration 无法表达 zigzag ownership，导致普通 rank-order gather 不能还原原序列。它有双进程 CPU/Gloo 真实源码复现，但没有 CUDA/NCCCL 专项复现。该问题不应并入 CC12 selector root cause，也不应归因到 llm-train API 使用错误。

图表计划：图 4：三条调查线的证据状态图，分别标记 confirmed、withdrawn 和 separately confirmed。

11 形式化验证在这次事件中起了什么作用

本节任务：既不夸大 Lean 直接发现 kernel bug，也不忽略严格 authority policy 的作用。

TrainVerify 的 Lean theorem 没有直接证明 selector 条件错误。真正产生价值的是验证链对输入真实性的要求：

- production authority 必须来自真实双 GPU、双 rank 路径，CPU smoke 不能替代；
- hardware、profile、patched source 和 graph bytes 必须进入 provenance；
- 失败不能通过手写图、弱化命题或修改 metadata 绕过；
- 修复后必须重新生成 authority，再重新 emission 和证明。

由此可形成更一般的系统论点：proof-oriented validation 会把“测试环境没有覆盖的编译路径”变成必须执行和记录的前置条件。但这一论点需要更多案例与对照实验，不能仅靠单事件扩大成普遍结论。

12 预防措施与后续行动

本节任务：把教训转成 owner 明确、可验收的工程动作。

行动	责任层	验收标准	当前状态
用能力表替代开放式 CC >= N 分派	llm-train	未知新 CC fail closed 或使用明确 generic path	待上游设计
增加 CC12 selector 单测	llm-train	两处 grouped-GEMM path 都选 generic candidates	待补
增加真实 CC12 双 rank CI	基础设施	production compile 和最小 runtime 测试通过	受硬件资源约束
记录 candidate rejection 原因	runtime	failure 输出候选、backend 与首个原因	待补
补充性能回归	llm-train	编译与 runtime 指标可复核	待测
保留 patch provenance	TrainVerify	source hash、patch ID、rank receipt 一致	已实现
撤回结论登记	TrainVerify/docs	候选 revision 不能进入 production allowlist	已实现

13 对 OSDI/SOSP 投稿的作用

本节任务：决定这份材料是独立事件论文、主论文案例，还是 artifact appendix。

单独的两行 selector 修复不足以构成 OSDI/SOSP 论文。更有研究价值的角度是：严格的 proof/authority pipeline 如何暴露传统 smoke test 未覆盖的跨层错误，并如何用 provenance、防绕过和撤回机制约束诊断。是否能独立成文取决于：

- 是否有多个跨模型、编译器和硬件的案例；
- 是否能与普通 CI、differential testing 和 compiler self-check 做系统对比；
- 是否有结构化诊断方法，而不只是事后复盘；
- 是否有量化的检测率、定位成本和误报/撤回分析。

在证据不足时，这份报告更适合作为 TrainVerify 主论文的详细 case study 与 artifact appendix，同时单独公开为技术事件报告。

14 相关工作计划

围绕 GPU kernel dispatch、architecture capability detection、Triton autotuning、ML compiler testing、translation validation、postmortem methodology 和 proof-producing systems 组织。精确比较必须查阅原始论文与上游 issue/commit，提纲阶段不写未经核验的优先权结论。

15 结论

结论保持三句话：发生的是 CC12 grouped-GEMM selector 兼容问题；最小补丁恢复了真实双 rank authority path，并被 provenance 绑定；该事件没有证明 llm-train 的训练数学语义错误。随后指出，production-path evidence 和可撤回的诊断记录是 TrainVerify 方法的重要组成部分。

A 计划附录

- A：完整环境与 revision 清单。
- B：原始失败日志和符号化 stack trace。
- C：selector 源码、最小补丁与精确 hash。
- D：补丁脚本的 fail-closed 测试，并区分已存在测试与待补测试。
- E：双 rank 复现命令、rank receipts 和 authority manifest。
- F：性能与跨硬件回归。
- G：K/V 与 balance-loss 撤回的证据时间线。
- H：nnScaler zigzag/RVD 独立 finding 的复现边界。

成稿前的 BLOCK

- 恢复或明确缺失首次 production 失败的原始日志、时间戳和完整环境。
- 用源码和日志证明“B200-only candidate set”与失败之间的确切关系，不从成功补丁反推未记录细节。
- 运行补丁前后可比较的性能实验；没有数据时不得声称无性能影响。
- 检查 llm-train 上游当前版本是否已修复，并限定受影响 revision 范围。
- 验证 CC10、CC12 和较低 CC 路径，区分真实硬件测试与静态 selector 测试。
- 为所有图、表和结论建立 claim-to-evidence ledger。